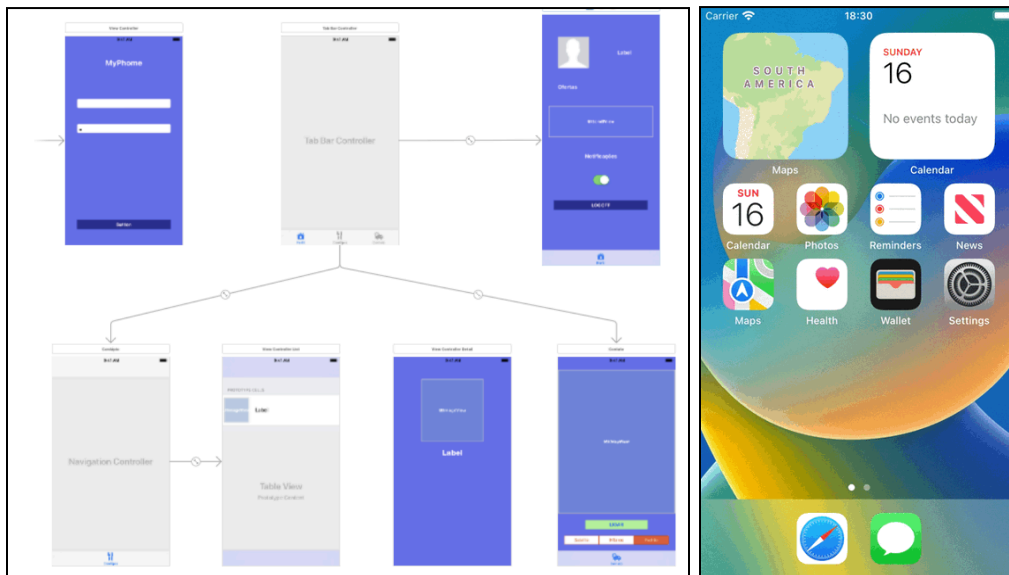


Desenvolvimento Web e Mobile - Full Stack

<https://pos.ufg.br/p/desenvolvimento-web-mobile-full-stack>

Programação Mobile com iOS - Prof. Silvano Malfatti



1. Desenvolvimento de aplicações com UIKit

O UIKit é um framework responsável por fornecer a interface gráfica e elementos de interação para aplicações iOS e iPadOS. Ele fornece componentes essenciais para a criação de interfaces responsivas e interativas.

Principais funcionalidades do UIKit:

- Interface de Usuário: Fornece elementos como UIView, UIButton, UILabel, UITextField, UIImageView, entre outros, para construção da interface gráfica.
- Gerenciamento de Layout: Utiliza Auto Layout e Constraints para criar interfaces responsivas em diferentes tamanhos de tela.

- Navegação e Controle de Fluxo: UINavigationController, UITabBarController e UIPageViewController permitem criar estruturas de navegação intuitivas.
- Toques e Gestos: UITapGestureRecognizer, UIPanGestureRecognizer e outros são usados para capturar interações do usuário.
- Animações e Gráficos: UIView.animate e Core Animation permitem criar animações fluidas na interface.
- Gerenciamento de Texto e Teclado: UITextView, UITextField e UIKeyboard fornecem suporte para entrada de texto e personalização do teclado virtual.
- Suporte a Multitarefa e Notificações: UIApplication gerencia o ciclo de vida do aplicativo e suas interações com o sistema operacional.

O UIKit é o framework principal para criação de interfaces no iOS e iPadOS, permitindo que os desenvolvedores criem experiências fluidas e interativas para os usuários.

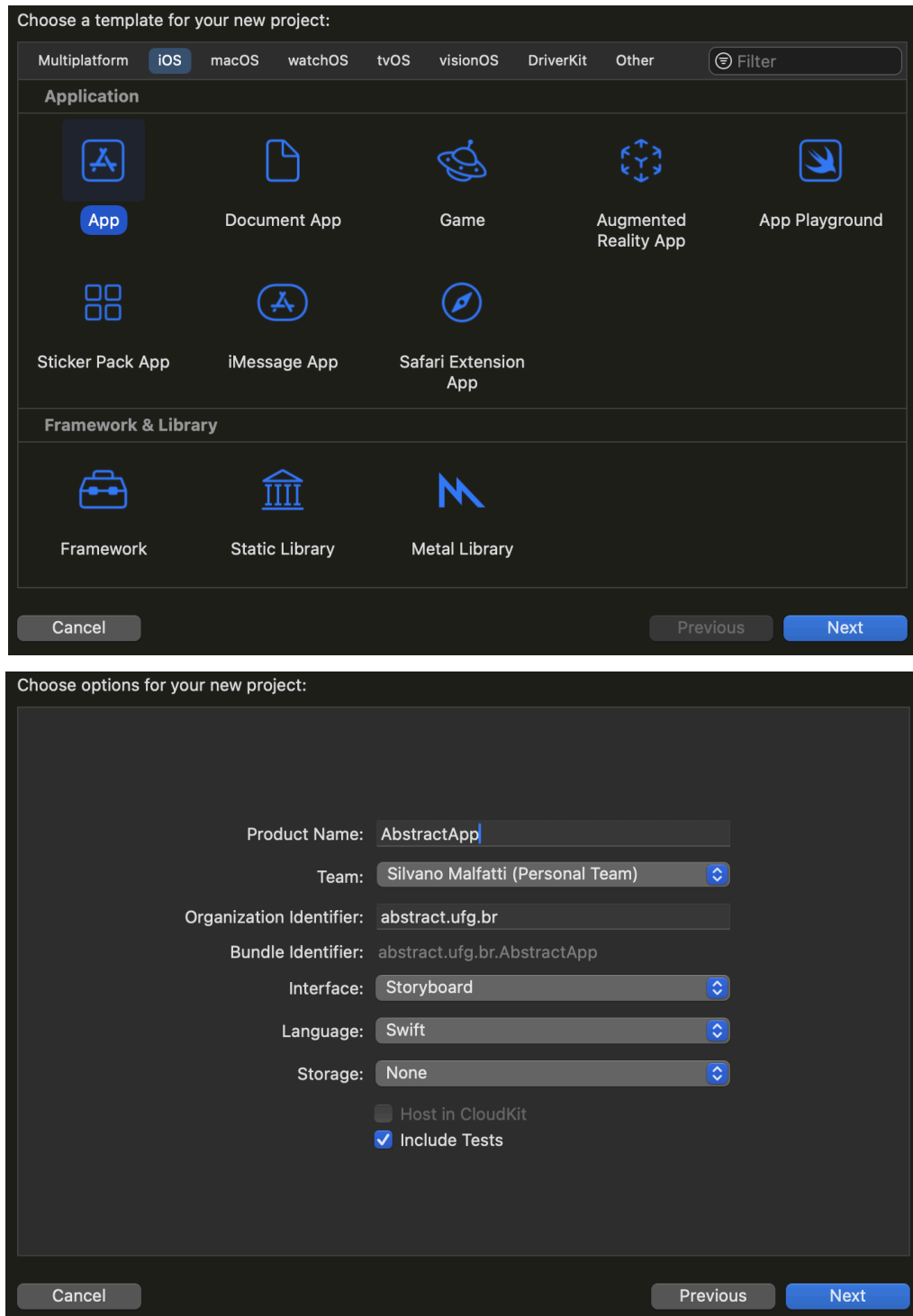


Figura 1 - Criando o primeiro projeto.

1. Product Name (Nome do Produto)

Nome do seu aplicativo. Esse será o nome do projeto e o nome que aparecerá no Xcode.

Esse nome também será usado para criar o identificador do bundle.

2. Team (Equipe)

Escolha a equipe de desenvolvimento associada à sua conta Apple Developer.

Se você não estiver inscrito no programa de desenvolvedores da Apple, pode selecionar None.

Necessário para testar apps em dispositivos físicos e para publicação na App Store.

3. Organization Name (Nome da Organização)

Nome da empresa ou do desenvolvedor responsável pelo app.

Esse nome não afeta diretamente o funcionamento do aplicativo, mas é usado para identificar a organização no código-fonte.

4. Organization Identifier (Identificador da Organização)

Geralmente segue o formato com.seuNome, com.suaEmpresa ou br.com.suaEmpresa.

Esse identificador será usado para compor o Bundle Identifier, que é único para cada aplicativo na App Store.

5. Bundle Identifier

Gerado automaticamente combinando o Organization Identifier com o Product Name.

Exemplo: com.empresa.meuapp

Esse identificador é único e usado para identificar o aplicativo na App Store e no sistema operacional.

6. Interface

Define a tecnologia usada para construir a interface do usuário.

Storyboard: Interface criada visualmente através do Interface Builder.

SwiftUI: Interface baseada em código declarativo com SwiftUI.

7. Language (Linguagem de prog. utilizada como base)

Escolha a linguagem de programação do projeto.

Swift: Recomendado para novos projetos.

Objective-C: Apenas para manutenção de projetos antigos.

8. Use Core Data

Se marcado, o Xcode incluirá configurações para persistência de dados usando Core Data.

Recomendado se o aplicativo precisar armazenar dados estruturados localmente.

9. Include Tests

Se marcado, adiciona arquivos para testes unitários e testes de UI.

Útil para garantir a qualidade do código e prevenir bugs.

10. Use CloudKit

Se marcado, adiciona suporte ao iCloud para armazenamento de dados na nuvem.

Útil para sincronização entre dispositivos.

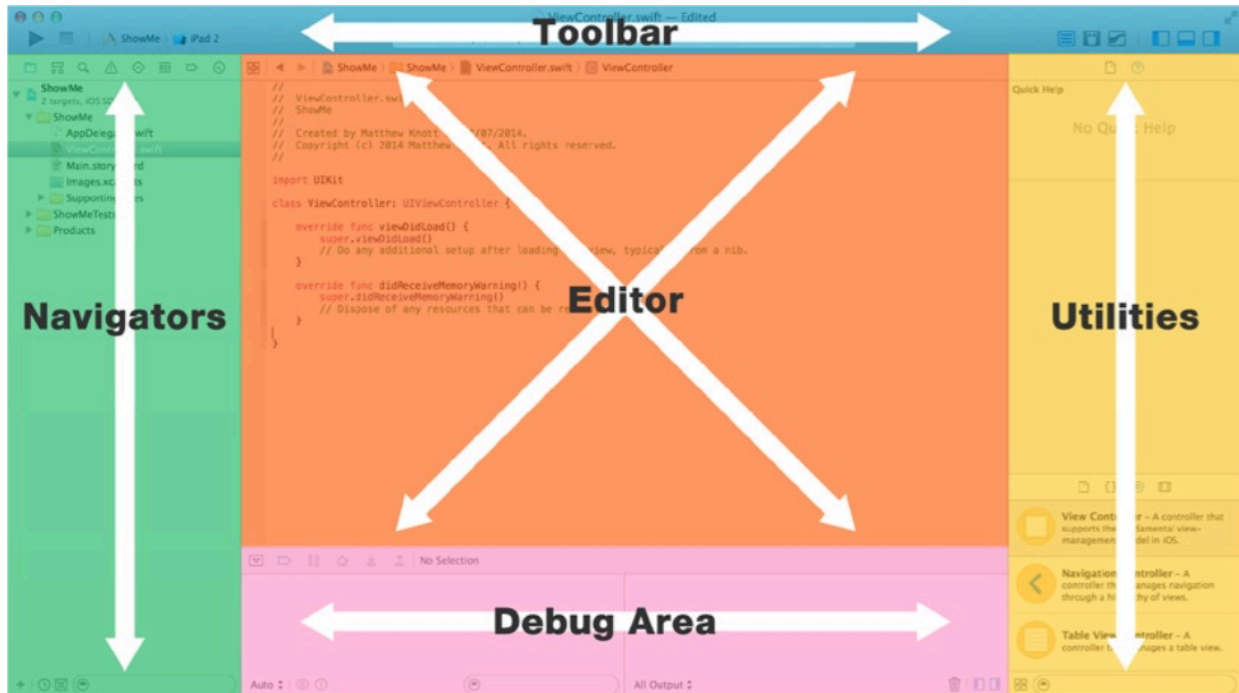


Figura 2 - Interface da IDE XCode.

2. Estrutura de um projeto iOS

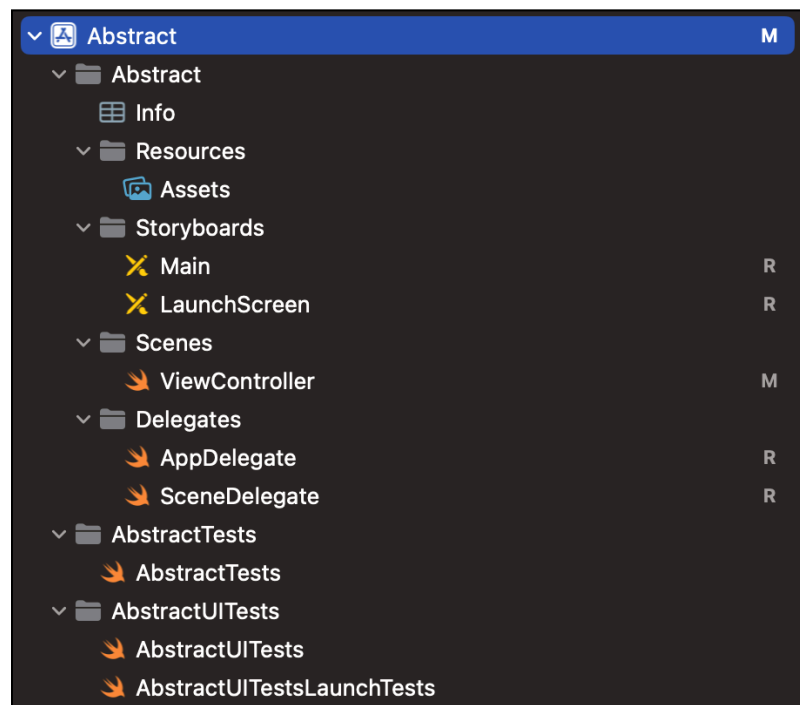


Figura 3 - Estrutura de um projeto iOS.

1. AppDelegate.swift (será descontinuado em breve)

Responsável pelo ciclo de vida do aplicativo e configurações iniciais.

Contém métodos como `application(_:didFinishLaunchingWithOptions:)`, que é chamado quando o app inicia.

Gerencia eventos do app como entrada em segundo plano ou término da execução.

2. SceneDelegate.swift

Responsável pelo gerenciamento de múltiplas cenas no iOS 13+ (ou seja, múltiplas janelas no iPad e suporte a multitarefa).

Controla a criação e configuração da `UIWindow` principal da aplicação.

3. ViewController.swift

Arquivo que contém a classe `ViewController`, que é responsável por controlar a tela principal do app.

Se comunica com o `Main.storyboard` e pode ser modificado para adicionar funcionalidades e lógica de interface.

4. Main.storyboard

Arquivo visual que contém a interface do usuário.

Inicialmente, contém apenas um `UIViewController` (a tela inicial do app).

Permite criar interfaces de forma gráfica, arrastando componentes.

5. Assets.xcassets

Diretório onde ficam os recursos visuais do aplicativo, como imagens e cores.

Contém por padrão:

`AppIcon` (ícones do aplicativo para diferentes resoluções).

`LaunchImage` (se aplicável).

6. LaunchScreen.storyboard

Define a tela de abertura (splash screen) do app antes da primeira tela ser carregada.

Inicialmente, contém apenas um `UIViewController` vazio.

7. Info.plist

Arquivo de configuração do aplicativo em formato XML.

Contém informações essenciais como:

Bundle Identifier

Display Name (Nome do aplicativo)

Supported Interface Orientations (Orientações de tela suportadas)

Privacy Permissions (Permissões para câmera, localização, etc.)

3. Máquina de estados de cena para um aplicativo iOS

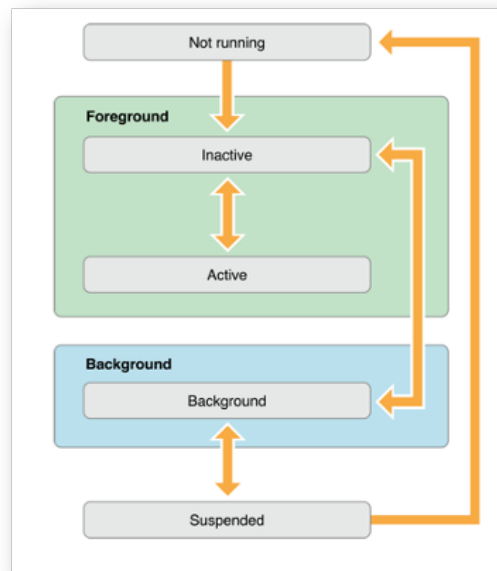


Figura 4 - Máquina de estados de uma cena iOS.


```
Abstract > Abstract > Delegates > SceneDelegate > sceneDidEnterBackground(_:)
10 class SceneDelegate: UIResponder, UIWindowSceneDelegate {
31 func sceneDidBecomeActive(_ scene: UIScene) {
35 }
36
37 func sceneWillResignActive(_ scene: UIScene) {
38     // Called when the scene will move from an active s
39     // This may occur due to temporary interruptions (e
40     print("Cena irá perder o foco")
41 }
42
43 func sceneWillEnterForeground(_ scene: UIScene) {
44     // Called as the scene transitions from the backgro
45     // Use this method to undo the changes made on ente
46     print("Cena irá ganhar o foco")
47 }
```

Aplicação carregamento finalizado
Cena criada e configurada
Cena será conectada à janela
Cena irá ganhar o foco
Aplicação finalizada, cena destruída – chamado pelo iOS
Cena está ativa

Cena irá perder o foco

Cena está ativa

Cena irá perder o foco

Cena entrou em background

Cena irá ganhar o foco
Cena está ativa

Figura 5 - Máquina de estados de uma cena iOS.

4. Executando uma aplicação

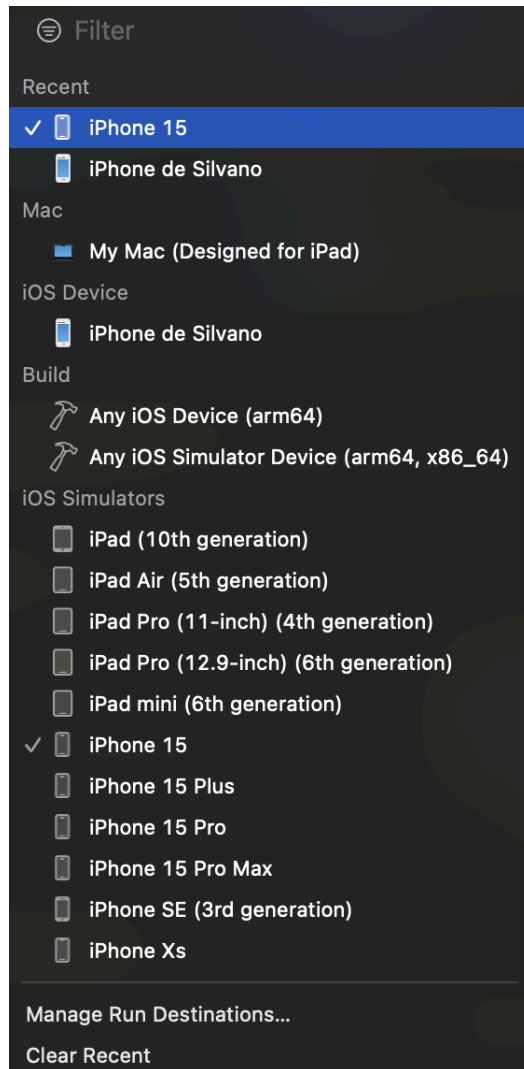


Figura 6 - Executando uma aplicação através do XCode.

5. Desenvolvendo Apps com StoryBoard

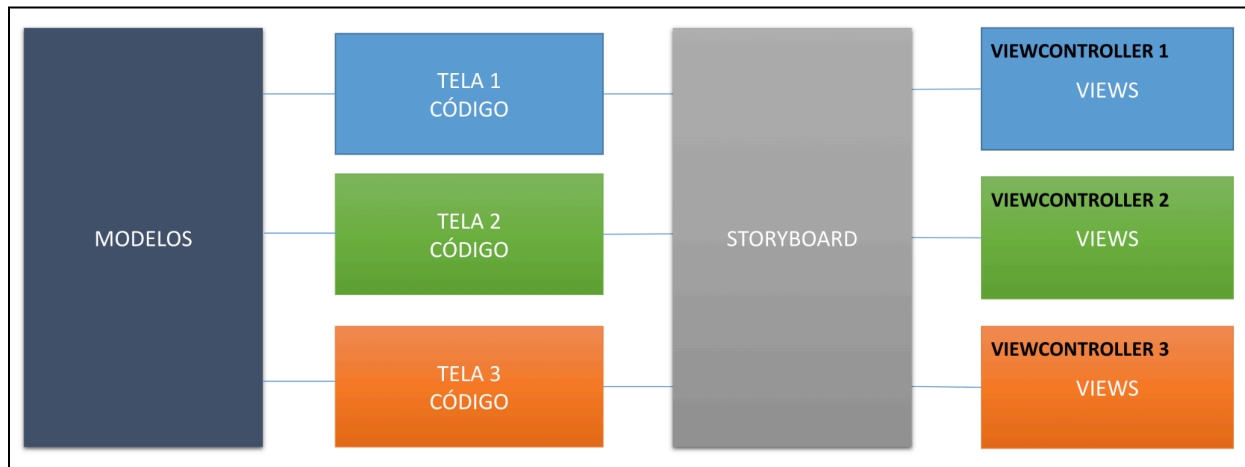


Figura 7 - Arquitetura MVC de uma aplicação baseada em Storyboard.

5.1 Máquina de estados de uma ViewController

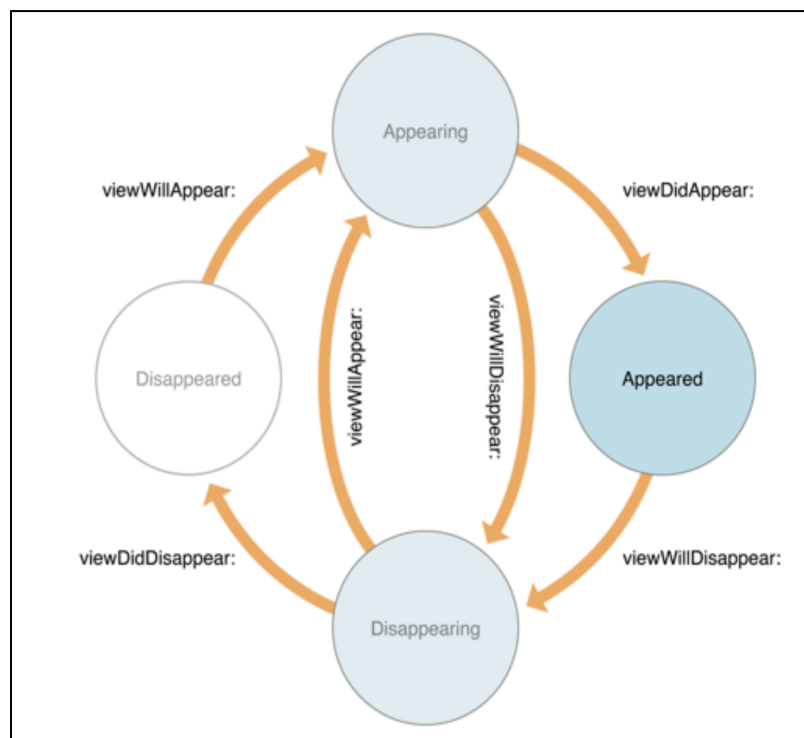


Figura 8 - Máquina de estados de uma ViewController.

5.2 Componentes Visuais (Views)

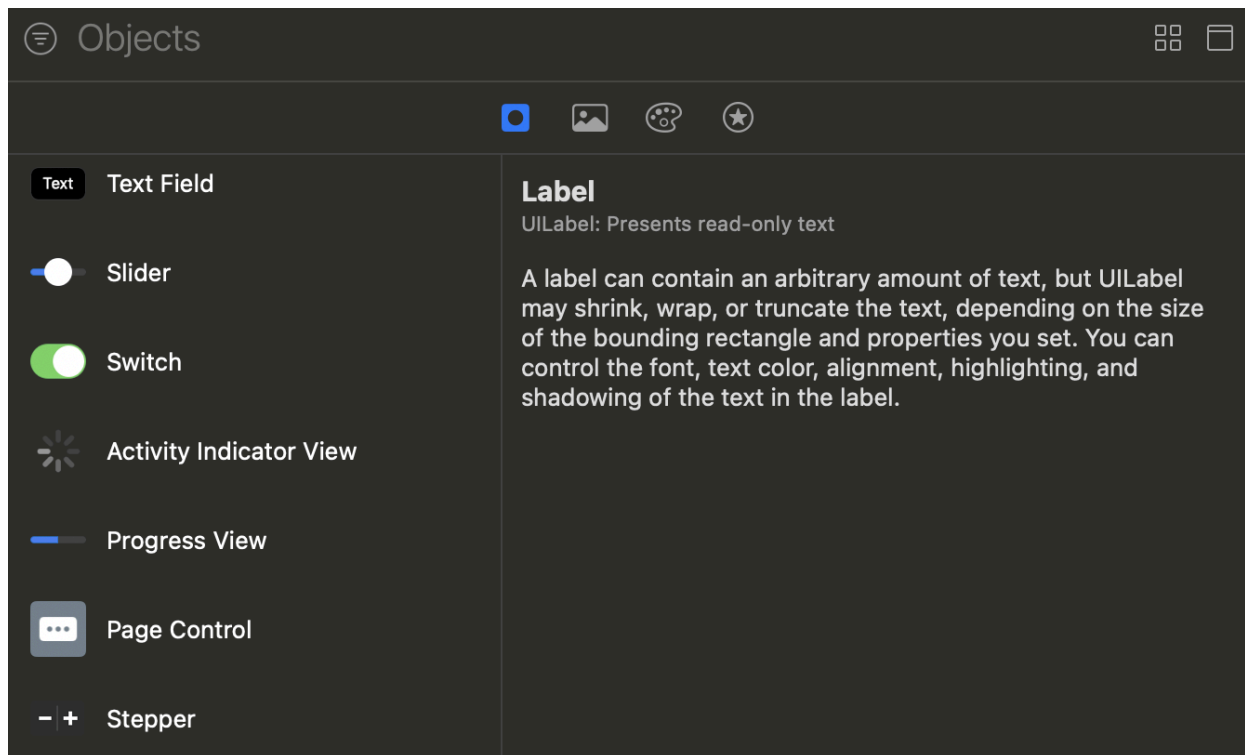


Figura 9 - Biblioteca de componentes visuais.

5.3 Alterando a interface de LaunchScreen

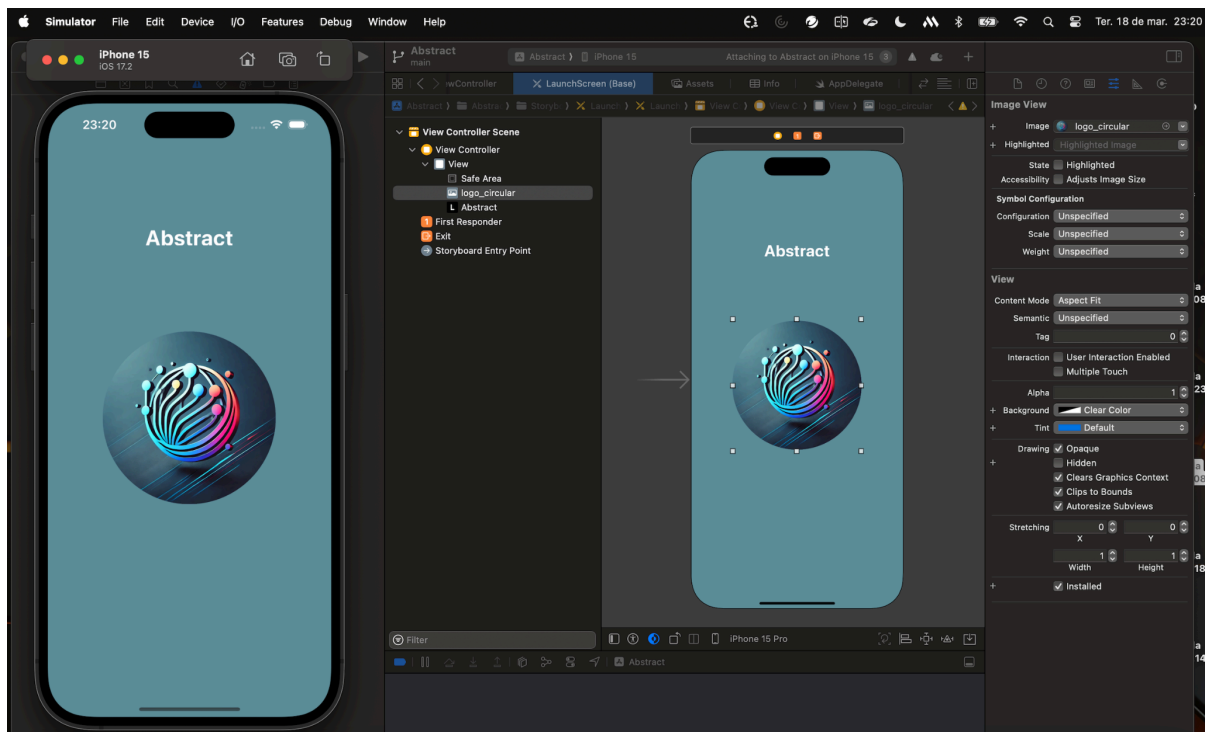


Figura 10 - Alterando a interface do LaunchScreen.



Cuidado para não alterar o Storyboard principal (Main)



5.4 Alterando a interface da tela de Login

- Renomear o arquivo padrão para Viewcontroller (verificar relacionamento com StoryBoard)
- Inserir Views e configurando suas propriedades
- Criar os relacionamentos entre os componentes e o código da ViewController. (Configurar propriedades e eventos)
- Atribuir regras de layout
- Criar arquivos de internacionalização
- Implementar a busca por dados do login
- Implementar a persistência em caso de opção do usuário
- Implementar a troca de contexto

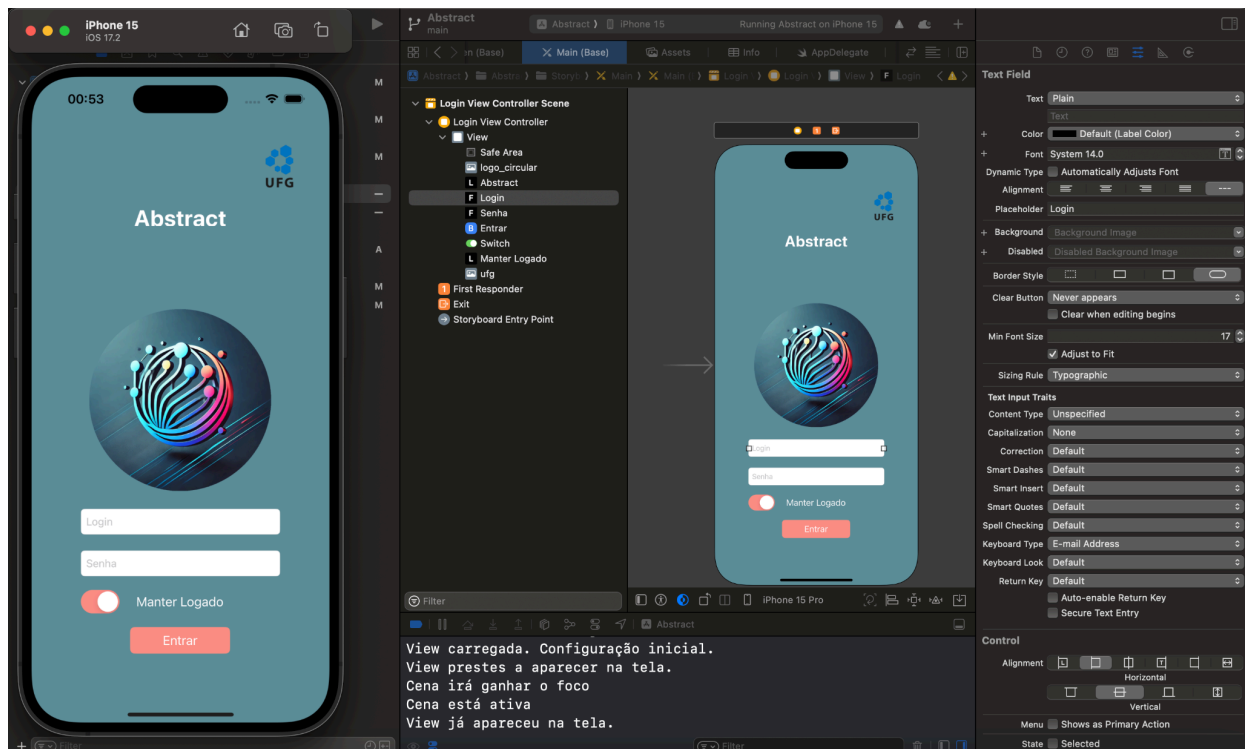


Figura 11 - Tela de Login Abstract App.

5.5 Trocas de contexto

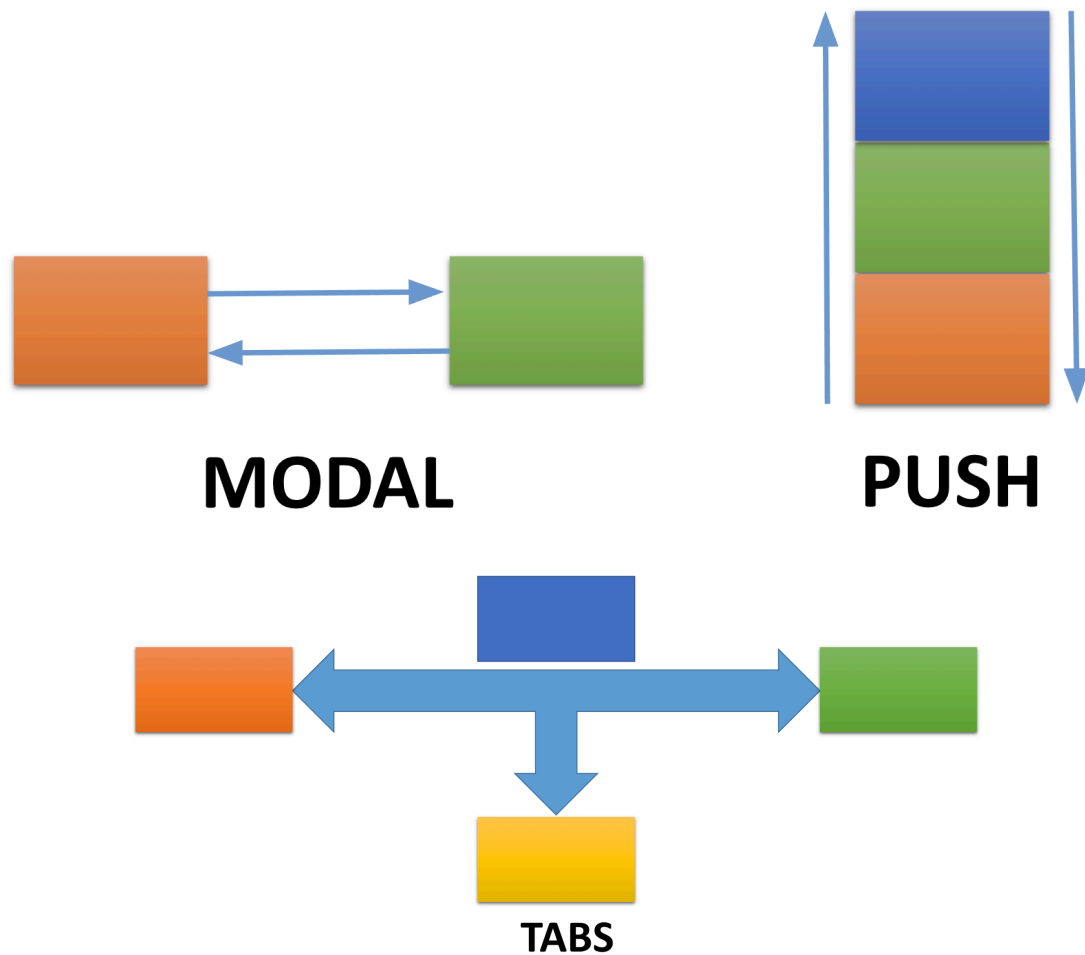


Figura 12 - Tipos de trocas de contexto em iOS.

- **Modal Presentation (Apresentação Modal)**

Uso: Quando se quer apresentar uma nova tela sem relação hierárquica com a anterior (ex: configurações, perfil, alertas).

Como funciona: A nova ViewController é apresentada por cima da atual.

Exemplo:

```
present(profile, animated: true)
```

Reversão: `dismiss(animated: true)`

Tipos de Modal Presentation:

`.fullScreen` (padrão antigo)

`.pageSheet`, `.formSheet`, `.automatic` (novos com iOS 13+)

`.popover` (em iPadOS ou com setas)

- **Push Navigation (via UINavigationController)**

Uso: Quando há uma hierarquia de telas (ex: lista → detalhes).

Como funciona: A nova tela é empurrada (push) na pilha de navegação.

Exemplo:

```
navigationController?.pushViewController(detailVC, animated: true)
```

Reversão: Via `popViewController`.

- **Container View Controllers**

Uso: Para exibir múltiplas telas dentro de uma tela principal (ex: `UITabBarController`, `UIPageViewController`, `UISplitViewController`).

Controle da troca: Via APIs dos próprios containers.

6 Consumo de serviços

O que são Codable, Decodable e Encodable?

Decodable

- Permite transformar JSON → objeto Swift.
- Ex: pegar os dados de uma API e transformá-los em uma struct.

Encodable

- Permite transformar objeto Swift → JSON.
- Útil ao enviar dados para a API (ex: criar usuário, login, etc).

Codable = Decodable & Encodable

- Um atalho que diz: "essa struct pode ser codificada e decodificada".

6.1 Relação com Consumo de API REST

Quando você faz uma requisição para um serviço REST, normalmente o servidor responde com um JSON, e você quer transformar esse JSON em um modelo Swift para trabalhar mais facilmente. É aqui que Decodable brilha.

Exemplo real: Consumo de um JSON de usuário

Suponha que a API retorne isso:

```
{  
  "name": "João Silva",  
  "email": "joao@example.com",  
  "phone": "+55 11 91234-5678"  
}
```

Crie a struct Swift com Codable:


```
struct User: Codable {  
    let name: String  
    let email: String  
    let phone: String  
}  
  
guard let url = URL(string: "https://api.exemplo.com/usuario") else { return }  
  
URLSession.shared.dataTask(with: url) { data, response, error in  
    if let data = data {  
        do {  
            let user = try JSONDecoder().decode(User.self, from: data)  
            print("Nome: \(user.name)")  
        } catch {  
            print("Erro ao decodificar: \(error)")  
        }  
    }  
}.resume()
```



E para enviar dados (usando Encodable):

```
let newUser = User(name: "Maria", email: "maria@example.com", phone: "1199999999")  
let jsonData = try? JSONEncoder().encode(newUser)
```

7 Desenvolvimento com ViewCode

ViewCode é uma abordagem de desenvolvimento iOS onde todas as interfaces de usuário são criadas via código, sem o uso de Storyboards ou XIBs. É comum combinar isso com boas práticas como Auto Layout programático, protocolos de organização, e frameworks como SnapKit.

Por que usar ViewCode?

- Maior controle e flexibilidade sobre a UI.
- Melhor versionamento com Git (sem conflitos de XML de Storyboard).
- Mais fácil de reutilizar componentes.
- Testabilidade melhor (view isolada, sem dependência de Interface Builder).

Organização com ViewCodable

Embora não seja uma API oficial da Apple, o termo ViewCodable costuma ser usado em projetos que adotam uma estrutura limpa e padronizada para construir views. Geralmente envolve o uso de protocolos como:

```
protocol ViewCodable {  
    func buildViewHierarchy()  
    func setupConstraints()  
    func setupAdditionalConfiguration()  
    func setupView()  
}
```

Exemplo de desenvolvimento de uma View com ViewCode:

```

//
// TimelineLoadingView.swift
// AbstractApp
//
// Created by Silvano Maneck Malfatti on 05/04/25.
//

import UIKit
import SnapKit

class TimelineLoadingView: UIView {

    // MARK: - Views
    private let loadingLabel: UILabel = {
        let label = UILabel()
        label.text = "Carregando..."
        label.font = .systemFont(ofSize: 18)
        label.textAlignment = .center
        return label
    }()

    private let loadingIndicator: UIActivityIndicatorView = {
        let indicator = UIActivityIndicatorView(style: .large)
        indicator.color = .gray
        indicator.startAnimating()
        return indicator
    }()

    // MARK: - Initializers
    override init(frame: CGRect) {
        super.init(frame: frame)
        buildLayout()
    }

    required init?(coder: NSCoder) {
        fatalError("esta view nao pode ser inicializada")
    }
}

extension TimelineLoadingView: ViewCodable {
    func buildViewHierarchy() {
        addSubview(loadingLabel)
        addSubview(loadingIndicator)
    }
}

```

```

    }

    func buildViewConstraints() {
        loadingLabel.snp.makeConstraints { make in
            make.top.equalToSuperview().offset(40)
            make.centerX.equalToSuperview()
        }

        loadingIndicator.snp.makeConstraints { make in
            make.centerX.centerY.equalToSuperview()
            make.width.height.equalTo(50)
        }
    }

    func additionalConfig() {
        self.backgroundColor = .white
    }
}

```

Vantagens dessa abordagem:

- Código modular e organizado.
- Fácil leitura e manutenção.
- Componentização facilita testes e reaproveitamento.
- Integra bem com SnapKit, Combine, SwiftUI wrappers, entre outros.

7 Desenvolvimento XIB

Um XIB (também chamado de Nib) é um arquivo de interface visual criado com o Interface Builder do Xcode. Ele representa uma única view ou componente de interface, podendo ser reutilizado em diferentes partes do app.

Quando usar XIB?

- Quando você quer criar componentes reutilizáveis com visual pré-definido.

- Se quiser evitar Storyboards gigantes e ainda usar interface visual.
- Para montar células customizadas (UITableViewCell, UICollectionViewCell) com menos código.

Onde usar?

- Header/Footer views
 - Views reutilizáveis
 - Custom cells
 - Pequenos componentes visuais com layout fixo
-

Vantagens

- Interface visual rápida de montar.
- Reduz código para pequenos componentes.
- Facilita colaboração com designers (via Interface Builder).

Desvantagens

- Pouca flexibilidade para views dinâmicas.
- Carregamento de XIBs exige atenção (pode falhar silenciosamente).
- Menor testabilidade comparado com ViewCode.

8 Desenvolvimento com SwiftUI

SwiftUI é o framework moderno da Apple para construir interfaces declarativas

em iOS, macOS, watchOS e tvOS. Ele foi introduzido em 2019 e tem como foco simplicidade, reatividade e declaração de UI via código Swift puro.

Características principais:

- Declarativo: você descreve o que a interface deve mostrar, não como renderizá-la.
 - Reativo: qualquer mudança no estado da view automaticamente reflete na interface.
 - Multiplataforma: um único código pode rodar em iOS, iPadOS, macOS, watchOS e tvOS.
 - Integração com Combine: responde a mudanças de estado e eventos assíncronos com facilidade.
 -
 - Código **limpo e conciso**
 - Interface **visual no Xcode (Canvas)**
 - Fácil de **testar e iterar rapidamente**
 - Total integração com **dados reativos**
-

Limitações (em apps mais complexos):

- Algumas APIs ainda são mais maduras no UIKit.
- Integração com `UIViewController`, `UIKit` e bibliotecas legadas pode exigir bridges.
- SwiftUI ainda está evoluindo – novos recursos vêm a cada versão do iOS.

Exemplo criação de uma ViewController e View com Swift UI.

```
import SwiftUI

struct RegistrationFormView: View {
    @State private var name: String = ""
    @State private var email: String = ""
    @State private var password: String = ""
    @Environment(\.presentationMode) var presentationMode

    var body: some View {
        NavigationView {
            Form {
                Section(header: Text("Informações Pessoais")) {
                    TextField("Nome", text: $name)
                    TextField("Email", text: $email)
                        .keyboardType(.emailAddress)
                    SecureField("Senha", text: $password)
                }

                Button("Cadastrar") {
                    print("Nome: \(name)")
                    print("Email: \(email)")
                    print("Senha: \(password)")
                    presentationMode.wrappedValue.dismiss()
                }
            }
        }
        .navigationTitle("Cadastro")
    }
}
```

```
import UIKit
import SwiftUI

class CreateAccountViewController: UIViewController {

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .white
        title = "Criar Conta"
    }
}
```

```
let openSwiftUIButton = UIButton(type: .system)
openSwiftUIButton.setTitle("Abrir Formulário SwiftUI", for: .normal)
openSwiftUIButton.addTarget(self, action: #selector(openSwiftUIForm), for:
.touchUpInside)
openSwiftUIButton.translatesAutoresizingMaskIntoConstraints = false

view.addSubview(openSwiftUIButton)
NSLayoutConstraint.activate([
    openSwiftUIButton.centerXAnchor.constraint(equalTo: view.centerXAnchor),
    openSwiftUIButton.centerYAnchor.constraint(equalTo: view.centerYAnchor)
])
}

@objc func openSwiftUIForm() {
    let swiftUIView = RegistrationFormView()
    let hostingController = UIHostingController(rootView: swiftUIView)
    present(hostingController, animated: true, completion: nil)
}
}
```